Pandas from Beginer to Advance level

Import Libaries

In [187…
```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine, text
```

In [2]:
```
data = {
    "name" : ["summit","rahul","ankit"],
    "age" : [34,33,30],
    "salary" : [25000, 30000, 35000]
}
```

In [3]:
```
df = pd.DataFrame(data)
df
```

Out[3]:

	name	age	salary
0	summit	34	25000
1	rahul	33	30000
2	ankit	30	35000

Load data

In [4]:
```
file_path = (r"C:\Users\aravit01\Downloads\orders_ab_YT.txt")
df = pd.read_csv(file_path)
```

In [5]:
```
df.head()

df.tail()

# returns sample/random 5 rows, axis indicates rows or columns
df.sample(5, axis= 0)
```

Out[5]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	catego
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furnitu
23	US-2021-156909	16-07-2021	18-07-2021	Second Class	SF-20065	East	FUR-CH-10002774	Furnitu
48	CA-2020-169194	20-06-2020	25-06-2020	Standard Class	LH-16900	East	TEC-PH-10003988	Technolo
34	CA-2021-107727	19-10-2021	23-10-2021	Second Class	MA-17560	Central	OFF-PA-10000249	Offi Suppli
11	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	TEC-PH-10002033	Technolo

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   order_id     50 non-null     object 
 1   order_date   49 non-null     object 
 2   ship_date    50 non-null     object 
 3   ship_mode    49 non-null     object 
 4   customer_id  50 non-null     object 
 5   region       50 non-null     object 
 6   product_id   50 non-null     object 
 7   category     50 non-null     object 
 8   sales        48 non-null     float64
 9   quantity     50 non-null     int64  
 10  profit       50 non-null     float64
dtypes: float64(2), int64(1), object(8)
memory usage: 4.4+ KB
```

stastical information for numeric cols

In [7]:
```
# applicable only for numerical cols, provides statistical information of all numer
df.describe()
```

Out[7]:

	sales	quantity	profit
count	48.000000	50.000000	50.000000
mean	288.966667	3.920000	-20.550000
std	551.893154	1.977836	251.767027
min	2.500000	2.000000	-1665.100000
25%	19.250000	2.000000	2.500000
50%	70.100000	3.000000	8.350000
75%	212.450000	5.000000	17.650000
max	3083.400000	9.000000	240.300000

In [8]:
```
df.columns
```

Out[8]:
```
Index(['order_id', 'order_date', 'ship_date', 'ship_mode', 'customer_id',
       'region', 'product_id', 'category', 'sales', 'quantity', 'profit'],
      dtype='object')
```

In [9]:
```
# header == None indicates, pandas assumes there is no header

df = pd.read_csv(file_path, header= None)
```

In [10]:
```
df = pd.read_csv(file_path)
df.head()
```

Out[10]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

In [11]:
```
col = ["order_id","order_date","region", "profit"]
```

```
df[col].head()

df[["order_id","order_date","region", "profit"]].head()
```

Out[11]:

	order_id	order_date	region	profit
0	CA-2020-152156	08-11-2020	South	41.9
1	CA-2020-152156	08-11-2020	South	219.6
2	CA-2020-138688	12-06-2020	West	6.9
3	US-2019-108966	11-10-2019	South	-383.0
4	US-2019-108966	NaN	South	2.5

In [12]:
```
# provide data type information
# multiple cols --> dataframe
# single col --> series

type(df)

type(df['order_id'])
```

Out[12]:
```
pandas.core.series.Series
```

Filtering data

In [13]:
```
# selection of particular index/ rows based on index

# select only index (0,4,10)
df.iloc[[0,4,10]]

# selection range of index ( 0 to 4) --> 4th index is not returned using iloc
df.iloc[:4]

# select range from index (39 to 43) & all cols
df.iloc[39:43]

# select only 2 factor index's & all cols
df.iloc[::2]

# select only 3 factor index's & all cols
df.iloc[::3]

# select only 4 factor index's & all cols
df.iloc[::4].head()
```

Out[13]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies
8	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	OFF-BI-10003910	Office Supplies
12	CA-2021-114412	15-04-2021	20-04-2021	Standard Class	AA-10480	South	OFF-PA-10002365	Office Supplies
16	CA-2018-105893	11-11-2018	18-11-2018	Standard Class	PK-19075	Central	OFF-ST-10004186	Office Supplies

In [14]:

```
# select all rows & cols
df.iloc[::].head()

# select first 10 rows & first 3 cols
df.iloc[0:11,0:4]

# select first 10 rows & (1,3,5) cols
df.iloc[0:11,[1,3,5]]

# select (3,10,15)row & (2,4,6) col
df.iloc[[3,10,15],[2,4,6]]

# select from 3row to end & 3 col to end col
df.iloc[3:,3:]

# select all rows & (last & last second) col
df.iloc[:,[-1,-2]]

# select last 10 rows & last 3 cols
df.iloc[-10:,-3:]
```

Out[14]:

	sales	quantity	profit
40	371.2	4	41.8
41	147.2	4	16.6
42	77.9	2	3.9
43	95.6	2	9.6
44	46.0	2	19.8
45	17.5	2	8.2
46	212.0	4	8.5
47	45.0	3	5.0
48	21.8	2	6.1
49	38.2	6	18.0

Difference iloc Vs loc

iloc (Integer Location) 1. Integer-based indexing → Selects rows/columns by position (0-based index). 2. End index is exclusive → When slicing, the last position is not included. 3. Works only with integers or integer lists/boolean masks. 4. Cannot use labels (column names or custom index labels) directly. loc (Label Location) 1. Label-based indexing → Selects rows/columns by labels (index name or column name). 2. End index is inclusive → When slicing, the last label is included. 3. Can use integers if your index labels are integers, but it will be label-based, not position-based. 4. Can also handle boolean masks and conditional filtering.

In [15]:
```
# select first 10 rows of order_id, region, sales

df.loc[0:10,['order_id','region','sales']]
```

Out[15]:

	order_id	region	sales
0	CA-2020-152156	South	NaN
1	CA-2020-152156	South	731.9
2	CA-2020-138688	West	14.6
3	US-2019-108966	South	957.6
4	US-2019-108966	South	22.4
5	CA-2018-115812	West	48.9
6	CA-2018-115812	West	7.3
7	CA-2018-115812	West	NaN
8	CA-2018-115812	West	18.5
9	CA-2018-115812	West	114.9
10	CA-2018-115812	West	1706.2

Indexing

In [16]:
```
df.set_index('order_id', inplace=True)
```

In [17]:
```
df.reset_index('order_id', inplace=True)
```

In [18]:
```
df.head()
```

Out[18]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

Sorting

In [19]:
```
df['profit'].sort_values(ascending=False).head(3)
```

Out[19]:
```
24    240.3
1     219.6
13    132.6
Name: profit, dtype: float64
```

In [20]:
```
df.sort_index(ascending=False).head()
```

Out[20]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	catego
49	CA-2019-115742	18-04-2019	22-04-2019	Standard Class	DP-13000	Central	OFF-BI-10004410	Offi Suppli
48	CA-2020-169194	20-06-2020	25-06-2020	Standard Class	LH-16900	East	TEC-PH-10003988	Technolo
47	CA-2020-169194	20-06-2020	25-06-2020	Standard Class	LH-16900	East	TEC-AC-10002167	Technolo
46	CA-2018-146703	20-10-2018	25-10-2018	Second Class	PO-18865	Central	OFF-ST-10001713	Offi Suppli
45	CA-2020-118255	11-03-2020	13-03-2020	First Class	ON-18715	Central	OFF-BI-10003291	Offi Suppli

In [21]:
```
# sorting based on single column

df.sort_values(by='sales', ascending=True)
df.sort_values(by='sales', ascending=False).head()
```

Out[21]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	catego
27	US-2019-150630	17-09-2019	21-09-2019	Standard Class	TB-21520	East	FUR-BO-10004834	Furnitu
10	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	FUR-TA-10001539	Furnitu
35	CA-2020-117590	08-12-2020	10-12-2020	First Class	GH-14485	Central	TEC-PH-10004977	Technolo
24	CA-2019-106320	25-09-2019	30-09-2019	Standard Class	EB-13870	West	FUR-TA-10000577	Furnitu
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furnitu

In [22]:
```
# sorting based on multiple column
# less quantity more profit

df.sort_values(by=['quantity','profit'], ascending=[True,False]).head()
```

Out[22]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	catego
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furnitu
44	CA-2020-118255	11-03-2020	13-03-2020	First Class	ON-18715	Central	TEC-AC-10000171	Technolo
17	CA-2018-167164	13-05-2018	15-05-2018	Second Class	AG-10270	West	OFF-ST-10000107	Offi Suppli
43	CA-2021-139619	19-09-2021	23-09-2021	Standard Class	ES-14080	South	OFF-ST-10003282	Offi Suppli
45	CA-2020-118255	11-03-2020	13-03-2020	First Class	ON-18715	Central	OFF-BI-10003291	Offi Suppli

Filtering data

In [23]:
```
# filtering data

df.head()
```

Out[23]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

In [24]:
```
# region is south & shi_mode is second_class

df[(df['region'] == "South") & (df['ship_mode'] == "Second Class")]
```

Out[24]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture

In [25]:
```
# how many unique order_id's profit > 50

df[df['profit'] > 50]['order_id'].nunique()
```

Out[25]: 5

In [26]:
```
# region is south & profit > 0

df[(df['region'] == 'South') & (df['profit'] > 0)]
```

Out[26]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies
12	CA-2021-114412	15-04-2021	20-04-2021	Standard Class	AA-10480	South	OFF-PA-10002365	Office Supplies
43	CA-2021-139619	19-09-2021	23-09-2021	Standard Class	ES-14080	South	OFF-ST-10003282	Office Supplies

In [27]:
```
# region is south or profit > 0

df[(df['region'] == 'South') | (df['profit'] > 0)][['order_id','order_date','ship_d
```

Out[27]:

	order_id	order_date	ship_date
0	CA-2020-152156	08-11-2020	11-11-2020
1	CA-2020-152156	08-11-2020	11-11-2020
2	CA-2020-138688	12-06-2020	16-06-2020
3	US-2019-108966	11-10-2019	18-10-2019
4	US-2019-108966	NaN	18-10-2019

In [28]:
```
# region is not south & profit > 0

df[~(df['region'] == 'South') & (df['profit'] > 0)].head()
```

Out[28]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
5	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	FUR-FU-10001487	Furniture
6	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	OFF-AR-10002833	Office Supplies
7	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	TEC-PH-10002275	Technology
8	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	OFF-BI-10003910	Office Supplies

In [29]:
```
# addition or updating column
```

In [30]:
```
df['country'] = 'India'
```

In [31]:
```
df['country'] = "USA"
```

In [32]:
```
df.head()
```

Out[32]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

In [33]:
```
df.drop(columns='country', inplace=True)
```

In [34]:
```
df.head()
```

Out[34]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

Function/condition based filtering

In [35]:
```
# function to return 0 or 1

def profit_flag(profit):
    if profit > 0:
        return 1
    else:
        return 0
```

In [36]:
```
# apply above function to profit_flag col based on profit col

df['profit_flag'] = df['profit'].apply(profit_flag)
```

In [37]:
```
# profit grouping

def profit_group(profit):
    if profit <= 0:
        return 'Loss'
    elif profit > 0 and profit <= 50:
        return 'Low Profit'
    else:
        return 'High Profit'
```

In [38]:
```
df['profit_group'] = df['profit'].apply(profit_group)
```

In [39]:
```
df['order_id_len'] = df['order_id'].apply(len)
```

In [40]:
```
df['category_upper'] = df['category'].str.upper()

df['category_lower'] = df['category'].str.lower()
```

Handling Missing values

In [41]:
```
# finding null values in df

df.info()

df.isna().sum()

df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 16 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   order_id        50 non-null     object 
 1   order_date      49 non-null     object 
 2   ship_date       50 non-null     object 
 3   ship_mode       49 non-null     object 
 4   customer_id     50 non-null     object 
 5   region          50 non-null     object 
 6   product_id      50 non-null     object 
 7   category        50 non-null     object 
 8   sales           48 non-null     float64
 9   quantity        50 non-null     int64  
 10  profit          50 non-null     float64
 11  profit_flag     50 non-null     int64  
 12  profit_group    50 non-null     object 
 13  order_id_len    50 non-null     int64  
 14  category_upper  50 non-null     object 
 15  category_lower  50 non-null     object 
dtypes: float64(2), int64(3), object(11)
memory usage: 6.4+ KB
```

Out[41]:
```
order_id          0
order_date        1
ship_date         0
ship_mode         1
customer_id       0
region            0
product_id        0
category          0
sales             2
quantity          0
profit            0
profit_flag       0
profit_group      0
order_id_len      0
category_upper    0
category_lower    0
dtype: int64
```

In [42]:
```
# finding & dropping null values

# check's all cols & drop rows
df.dropna()

# check's subset of cols & drop rows according
# based on order_date col, check any missig values are present, if yes then drop th
# cearly observe index 3,4 as null values & drop's accordingly
df.dropna(subset= ['order_date','ship_mode']).head()
```

Out[42]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
5	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	FUR-FU-10001487	Furniture
6	CA-2018-115812	09-06-2018	14-06-2018	Standard Class	BH-11710	West	OFF-AR-10002833	Office Supplies

In [43]:
```
# replace null's with na

# check's entier df, if any null value, replaces with NA
df.fillna('NA').head()
```

Out[43]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NA	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NA	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

Imputation of Null values for categorical values

In [242…
```
# specific cols
df['ship_mode'].fillna(df['ship_mode'].mode())

# forward fill
df['ship_mode'].ffill()

# backfill
df['ship_mode'].bfill().head()
```

Out[242…
```
0      Second Class
1      Second Class
2      Second Class
3    Standard Class
4    Standard Class
Name: ship_mode, dtype: object
```

In [45]:
```
data = {
    "name" : ["summit","rahul","ankit",'ankit','rahul'],
    "age" : [34,33,30,30,33],
    "salary" : [25000, 30000, 35000,35000,20000]
}
```

In [46]:
```
df_data = pd.DataFrame(data)
df_data
```

Out[46]:

	name	age	salary
0	summit	34	25000
1	rahul	33	30000
2	ankit	30	35000
3	ankit	30	35000
4	rahul	33	20000

In [47]:
```
# drop last row, based on pure duplicates
# default behaviour it keeps first row
df_data.drop_duplicates()

# drop first row, based on pure duplicates
df_data.drop_duplicates(keep= 'last')

# drop duplicates based on name,salary col
df_data.drop_duplicates(subset= ['name','salary'], keep= 'last')
```

Out[47]:

	name	age	salary
0	summit	34	25000
1	rahul	33	30000
3	ankit	30	35000
4	rahul	33	20000

casting datatypes

In [48]:
```
df.dtypes
```

Out[48]:
```
order_id           object
order_date         object
ship_date          object
ship_mode          object
customer_id        object
region             object
product_id         object
category           object
sales             float64
quantity            int64
profit            float64
profit_flag         int64
profit_group       object
order_id_len        int64
category_upper     object
category_lower     object
dtype: object
```

In [49]:
```
df['order_date'] = pd.to_datetime(df['order_date'], format = "%d-%m-%Y")
```

In [50]:
```
df['ship_date'] = pd.to_datetime(df['ship_date'], format = "%d-%m-%Y")
```

In [51]:
```
df['year'] = df['order_date'].dt.year
```

In [52]:
```
df['month'] = df['order_date'].dt.month
```

In [53]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 18 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   order_id        50 non-null     object
 1   order_date      49 non-null     datetime64[ns]
 2   ship_date       50 non-null     datetime64[ns]
 3   ship_mode       49 non-null     object
 4   customer_id     50 non-null     object
 5   region          50 non-null     object
 6   product_id      50 non-null     object
 7   category        50 non-null     object
 8   sales           48 non-null     float64
 9   quantity        50 non-null     int64
 10  profit          50 non-null     float64
 11  profit_flag     50 non-null     int64
 12  profit_group    50 non-null     object
 13  order_id_len    50 non-null     int64
 14  category_upper  50 non-null     object
 15  category_lower  50 non-null     object
 16  year            49 non-null     float64
 17  month           49 non-null     float64
dtypes: datetime64[ns](2), float64(4), int64(3), object(9)
memory usage: 7.2+ KB
```

In [54]:
```
min_order_year = df['order_date'].dt.year.min()
max_order_year = df['order_date'].dt.year.max()

df[df['order_date'].dt.year == max_order_year]
```

Out[54]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	catego
12	CA-2021-114412	2021-04-15	2021-04-20	Standard Class	AA-10480	South	OFF-PA-10002365	Offi Suppli
23	US-2021-156909	2021-07-16	2021-07-18	Second Class	SF-20065	East	FUR-CH-10002774	Furnitu
34	CA-2021-107727	2021-10-19	2021-10-23	Second Class	MA-17560	Central	OFF-PA-10000249	Offi Suppli
41	CA-2021-120999	2021-09-10	2021-09-15	Standard Class	LC-16930	Central	TEC-PH-10004093	Technolo
43	CA-2021-139619	2021-09-19	2021-09-23	Standard Class	ES-14080	South	OFF-ST-10003282	Offi Suppli

In [55]:
```
# days between 2 dates
```

```
df['date_diff'] = (df['ship_date'] - df['order_date']).dt.days

df['date_diff'] = (df['ship_date'] - df['order_date'])
```

Aggregation of data

In [56]:
```
## Aggregations for numeric cols

df['sales'].sum()

df['sales'].min()

df['sales'].max()

df['sales'].mean()

df['sales'].median()
```

Out[56]: 70.1

In [57]:
```
## Aggregations for categorical cols

df['category'].mode()

df['category'].unique()

df['category'].nunique()
```

Out[57]: 3

In [58]:
```
## group by operations
## group by single dimension (region)

df.groupby('region')['sales'].sum().reset_index(name= 'total_sales')
```

Out[58]:

	region	total_sales
0	Central	3824.8
1	East	3467.7
2	South	1823.1
3	West	4754.8

In [59]:
```
## group by multiple dimension (region,category)

grouped_df = df.groupby(['region','category'])['sales']\
                    .sum()\
                    .reset_index(name= 'total_sales')\
                    .sort_values(['region','total_sales'], ascending=[True,False])

grouped_df
```

Out[59]:

	region	category	total_sales
2	Central	Technology	1661.9
1	Central	Office Supplies	1227.5
0	Central	Furniture	935.4
3	East	Furniture	3279.0
4	East	Office Supplies	121.9
5	East	Technology	66.8
6	South	Furniture	1689.5
7	South	Office Supplies	133.6
8	West	Furniture	2799.7
10	West	Technology	1215.5
9	West	Office Supplies	739.6

In [60]:
```
## group by multiple dimension (region,category) & aggregation multiple measures

grouped_df_1 = df.groupby(['region','category']).agg({'sales':'sum','profit':'sum'}
                          .reset_index()\
                          .rename(columns={'sales':'total_sales','profit':'total_prof

grouped_df_1
```

Out[60]:

	region	category	total_sales	total_profit
0	Central	Furniture	935.4	-210.1
1	Central	Office Supplies	1227.5	-13.6
2	Central	Technology	1661.9	201.7
3	East	Furniture	3279.0	-1650.6
4	East	Office Supplies	121.9	1.5
5	East	Technology	66.8	11.1
6	South	Furniture	1689.5	-121.5
7	South	Office Supplies	133.6	17.5
8	West	Furniture	2799.7	339.8
9	West	Office Supplies	739.6	209.8
10	West	Technology	1215.5	186.9

In [61]:
```
# find region min_profit, max & total_profit by region
```

```
result_df_1 = (
    df.groupby('region')
      .agg(min_profit=('profit', 'min'),
           max_profit=('profit', 'max'),
           total_profit=('profit', 'sum'))
      .reset_index()
)

result_df_1
```

Out[61]:

	region	min_profit	max_profit	total_profit
0	Central	-148.0	123.5	-22.0
1	East	-1665.1	15.5	-1638.0
2	South	-383.0	219.6	-104.0
3	West	2.0	240.3	736.5

In [66]:
```
# find count records for each category

df['category'].value_counts().reset_index(name='count_records')
```

Out[66]:

	category	count_records
0	Office Supplies	28
1	Furniture	12
2	Technology	10

In [62]:
```
# find min & max salary by age

df_data.groupby('age').agg(
                        min_salary=('salary','min'),
                        max_salary=('salary','max'),
                        total_salary=('salary','sum')).reset_index()
```

Out[62]:

	age	min_salary	max_salary	total_salary
0	30	35000	35000	70000
1	33	20000	30000	50000
2	34	25000	25000	25000

Pivot & Unpivot data

In [83]:
```
## pivot table
## index >> rows
## columns >> column values
## values >> on which col aggregation is performed
```

```
## aggfunc >> type of aggregation (sum,min,max,mean)

pd.pivot_table(df,index='region', columns='category',values='sales', aggfunc='sum')
```

Out[83]:

region	Furniture	Office Supplies	Technology
Central	935.4	1227.5	1661.9
East	3279.0	121.9	66.8
South	1689.5	133.6	NaN
West	2799.7	739.6	1215.5

In [93]:

```
# pivot table with multiple aggregation

pd.pivot_table(df.sort_values(by='region',ascending=False),index='region', columns=
```

Out[93]:

	sum			min				C
category	Furniture	Office Supplies	Technology	Furniture	Office Supplies	Technology	Furniture	Sup
region								
Central	935.4	1227.5	1661.9	190.9	2.5	46.0	532.4	
East	3279.0	121.9	66.8	71.4	3.3	21.8	3083.4	
South	1689.5	133.6	NaN	731.9	15.6	NaN	957.6	
West	2799.7	739.6	1215.5	48.9	7.3	90.6	1706.2	

In [95]:

```
df_orders = pd.read_csv(file_path)
df_orders.head()
```

Out[95]:

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	category
0	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-BO-10001798	Furniture
1	CA-2020-152156	08-11-2020	11-11-2020	Second Class	CG-12520	South	FUR-CH-10000454	Furniture
2	CA-2020-138688	12-06-2020	16-06-2020	Second Class	DV-13045	West	OFF-LA-10000240	Office Supplies
3	US-2019-108966	11-10-2019	18-10-2019	NaN	SO-20335	South	FUR-TA-10000577	Furniture
4	US-2019-108966	NaN	18-10-2019	Standard Class	SO-20335	South	OFF-ST-10000760	Office Supplies

In [97]:
```
df_returns = pd.read_csv(r"C:\Users\aravit01\Downloads\returns_ab_YT.txt")
list_df_returns.head()
```

Out[97]:

	return_order_id	return_reason
0	CA-2018-105893	bad quality
1	CA-2018-143336	wrong item
2	CA-2018-146703	wrong item
3	CA-2018-167164	other
4	CA-2019-106320	bad quality

In [106…
```
unique_list = list(df_returns['return_order_id'].unique())
unique_list
```

Out[106…
```
['CA-2018-105893',
 'CA-2018-143336',
 'CA-2018-146703',
 'CA-2018-167164',
 'CA-2019-106320',
 'CA-2019-115742',
 'CA-2020-101343',
 'CA-2020-117590']
```

In [107…
```
df_orders[df_orders['order_id'].isin(unique_list)]
```

Out[107…

	order_id	order_date	ship_date	ship_mode	customer_id	region	product_id	catego
16	CA-2018-105893	11-11-2018	18-11-2018	Standard Class	PK-19075	Central	OFF-ST-10004186	Offi Suppli
17	CA-2018-167164	13-05-2018	15-05-2018	Second Class	AG-10270	West	OFF-ST-10000107	Offi Suppli
18	CA-2018-143336	27-08-2018	01-09-2018	Second Class	ZD-21925	West	OFF-AR-10003056	Offi Suppli
19	CA-2018-143336	27-08-2018	01-09-2018	Second Class	ZD-21925	West	TEC-PH-10001949	Technolo
20	CA-2018-143336	27-08-2018	01-09-2018	Second Class	ZD-21925	West	OFF-BI-10002215	Offi Suppli
24	CA-2019-106320	25-09-2019	30-09-2019	Standard Class	EB-13870	West	FUR-TA-10000577	Furnitu
35	CA-2020-117590	08-12-2020	10-12-2020	First Class	GH-14485	Central	TEC-PH-10004977	Technolo
36	CA-2020-117590	08-12-2020	10-12-2020	First Class	GH-14485	Central	FUR-FU-10003664	Furnitu
42	CA-2020-101343	17-07-2020	22-07-2020	Standard Class	RA-19885	West	OFF-ST-10003479	Offi Suppli
46	CA-2018-146703	20-10-2018	25-10-2018	Second Class	PO-18865	Central	OFF-ST-10001713	Offi Suppli
49	CA-2019-115742	18-04-2019	22-04-2019	Standard Class	DP-13000	Central	OFF-BI-10004410	Offi Suppli

Different Join/Merge operations

In [150…
```
inner_join_df = pd.merge(left=df_orders, right=df_returns[['return_order_id','retur
```

In [153…
```
left_join_df = pd.merge(left=df_orders, right=df_returns[['return_order_id','return
```

In [154…
```
right_join_df = pd.merge(left=df_orders, right=df_returns[['return_order_id','retur
```

In [155…
```
outer_join_df = pd.merge(left=df_orders, right=df_returns[['return_order_id','retur
```

In [ ]:
```
pd.merge()
```

In [156…
```
inner_join_df['return_reason'].value_counts().reset_index(name='count_records')
```

Out[156…

	return_reason	count_records
0	wrong item	6
1	bad quality	3
2	other	2

In [157…
```
print("no.of records on left table      >> ", df_orders.shape[0])
print("no.of records on right table     >> ", df_returns.shape[0])
print("no.of records on inner join      >> ", inner_join_df.shape[0])
print("no.of records on left join       >> ", left_join_df.shape[0])
print("no.of records on right join      >> ", right_join_df.shape[0])
print("no.of records on full outer join >> ", outer_join_df.shape[0])
```

```
no.of records on left table      >>  50
no.of records on right table     >>  8
no.of records on inner join      >>  11
no.of records on left join       >>  50
no.of records on right join      >>  11
no.of records on full outer join >>  50
```

In [158…
```
inner_join_df.groupby('return_reason')['sales'].sum().reset_index(name='returned_sa
```

Out[158…

	return_reason	returned_sales
0	bad quality	1788.4
1	other	93.7
2	wrong item	1745.2

In [160…
```
left_join_df['return_reason'] = left_join_df['return_reason'].fillna('Not returned'
```

In [161…
```
left_join_df.groupby('return_reason')['sales'].sum().reset_index(name='returned_sal
```

Out[161…

	return_reason	returned_sales
0	Not returned	10243.1
1	bad quality	1788.4
2	other	93.7
3	wrong item	1745.2

Union/concatenate operations

In [174…
```
## Union & Union All >> SQL function
## pd.concat >> pandas function

df1 = pd.DataFrame({'emp_id':[1,2,3,4],
                    'name':['name1','name2','name3','name4'],
                    'salary': [1000,2000,3000,4000]
                   })

df2 = pd.DataFrame({'emp_id':[5,6,7,8],
                    'name':['name5','name6','name7','name8'],
                    'salary': [5000,6000,7000,8000]
                   })
```

In [167…
```
df1,df2
```

Out[167…
```
(   emp_id   name  salary
 0       1  name1    1000
 1       2  name2    2000
 2       3  name3    3000
 3       4  name4    4000,
    emp_id   name  salary
 0       5  name5    5000
 1       6  name6    6000
 2       7  name7    7000
 3       8  name8    8000)
```

In [176…
```
df_final = pd.concat([df1,df2], ignore_index=True)
df_final
```

Out[176…

	emp_id	name	salary
0	1	name1	1000
1	2	name2	2000
2	3	name3	3000
3	4	name4	4000
4	5	name5	5000
5	6	name6	6000
6	7	name7	7000
7	8	name8	8000

Write back to different files

In [179…
```
## write back to csv/excel/parquet/db

df_final.to_csv('test_final_data', index= False)
```

In [182…
```
df_final.to_excel('test_final_data.xlsx')
```

In [183…
```
df_final.to_json('test_final_data.json')
```

In [185…
```
df_final.to_parquet('test_final_data')
```

Truncate & load data

In [ ]:
```
## Truncate and load data into table
```

In [ ]:
```
query = text("truncate table orders")

with engine.begin() as conn:
    conn.execute(query)
    conn.commit()
```

In [189…
```
## read data from sqldb
```

In [ ]:
```
query = text("select * from orders")

pd.read_sql(query, con=engine)
```

write data to sqldb

In [ ]:
```
# replace: Drop the table before inserting new values
# append: Insert new values to the existing table.

df.to_sql(name='orders', con=engine, schema='compute_ws_s', if_exists='replace')
```

Load data from Json

In [211…
```
import json

file = open(r"C:\Users\aravit01\Downloads\New Text Document.json", mode='r')
data = json.load(file)
data
```

Out[211…


```
[{'order_id': 1,
  'order_date': '2024-01-10',
  'total_amount': 200.5,
  'customer': {'customer_id': 101,
   'name': 'John Doe',
   'email': 'johndoe@example.com',
   'address': '123 Main StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 2},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 1}]},
 {'order_id': 2,
  'order_date': '2024-01-12',
  'total_amount': 150.0,
  'customer': {'customer_id': 102,
   'name': 'Jane Smith',
   'email': 'janesmith@example.com',
   'address': '456 Oak StSpringfield'},
  'products': [{'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1}]},
 {'order_id': 3,
  'order_date': '2024-01-12',
  'total_amount': 120.0,
  'customer': {'customer_id': 103,
   'name': 'Alice Johnson',
   'email': 'alicejohnson@example.com',
   'address': '789 Birch StSpringfield'},
  'products': [{'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 2}]},
 {'order_id': 4,
  'order_date': '2024-01-13',
  'total_amount': 300.5,
  'customer': {'customer_id': 101,
   'name': 'John Doe',
   'email': 'johndoe@example.com',
   'address': '123 Main StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 3},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
```

```
   'price': 45.0,
   'quantity': 2}]},
 {'order_id': 5,
  'order_date': '2024-01-14',
  'total_amount': 180.0,
  'customer': {'customer_id': 104,
   'name': 'Bob Brown',
   'email': 'bobbrown@example.com',
   'address': '101 Maple StSpringfield'},
  'products': [{'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1},
   {'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 1}]},
 {'order_id': 6,
  'order_date': '2024-01-14',
  'total_amount': 250.0,
  'customer': {'customer_id': 105,
   'name': 'Sania',
   'email': 'sania@example.com',
   'address': '102 Pine StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 2},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 2},
   {'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 1}]},
 {'order_id': 7,
  'order_date': '2024-01-16',
  'total_amount': 180.0,
  'customer': {'customer_id': 106,
   'name': 'Rohit',
   'email': 'rohit@example.com',
   'address': '103 Cedar StSpringfield'},
  'products': [{'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 2},
   {'product_id': 'P05',
    'name': 'Desk Organizer',
    'category': 'Furniture',
```

```
   'price': 30.0,
   'quantity': 1}]},
 {'order_id': 8,
  'order_date': '2024-01-16',
  'total_amount': 230.0,
  'customer': {'customer_id': 107,
   'name': 'Nina Patel',
   'email': 'ninapatel@example.com',
   'address': '104 Birch StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 3},
   {'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1}]},
 {'order_id': 9,
  'order_date': '2024-01-17',
  'total_amount': 310.0,
  'customer': {'customer_id': 108,
   'name': 'Tom Harris',
   'email': 'tomharris@example.com',
   'address': '105 Oak StSpringfield'},
  'products': [{'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 2},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 2}]},
 {'order_id': 10,
  'order_date': '2024-01-18',
  'total_amount': 130.0,
  'customer': {'customer_id': 109,
   'name': 'Eva Lee',
   'email': 'evalee@example.com',
   'address': '106 Elm StSpringfield'},
  'products': [{'product_id': 'P05',
    'name': 'Desk Organizer',
    'category': 'Furniture',
    'price': 30.0,
    'quantity': 1},
   {'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1}]},
 {'order_id': 11,
  'order_date': '2024-01-18',
  'total_amount': 250.0,
```

```
 'customer': {'customer_id': 101,
  'name': 'John Doe',
  'email': 'johndoe@example.com',
  'address': '123 Main StSpringfield'},
 'products': [{'product_id': 'P04',
   'name': 'Gaming Headset',
   'category': 'Electronics',
   'price': 60.0,
   'quantity': 2},
  {'product_id': 'P02',
   'name': 'Bluetooth Keyboard',
   'category': 'Electronics',
   'price': 45.0,
   'quantity': 2}]},
{'order_id': 12,
 'order_date': '2024-01-19',
 'total_amount': 180.0,
 'customer': {'customer_id': 103,
  'name': 'Alice Johnson',
  'email': 'alicejohnson@example.com',
  'address': '789 Birch StSpringfield'},
 'products': [{'product_id': 'P01',
   'name': 'Wireless Mouse',
   'category': 'Electronics',
   'price': 25.0,
   'quantity': 2},
  {'product_id': 'P05',
   'name': 'Desk Organizer',
   'category': 'Furniture',
   'price': 30.0,
   'quantity': 1}]}]
```

In [196…
```
type(data)
```

Out[196…
```
list
```

In [197…
```
data[0]
```

Out[197…
```
{'order_id': 1,
 'order_date': '2024-01-10',
 'total_amount': 200.5,
 'customer': {'customer_id': 101,
  'name': 'John Doe',
  'email': 'johndoe@example.com',
  'address': '123 Main StSpringfield'},
 'products': [{'product_id': 'P01',
   'name': 'Wireless Mouse',
   'category': 'Electronics',
   'price': 25.0,
   'quantity': 2},
  {'product_id': 'P02',
   'name': 'Bluetooth Keyboard',
   'category': 'Electronics',
   'price': 45.0,
   'quantity': 1}]}
```

Creating dimension & fact tables

In [ ]:
```
# orders_tbl
order_id, order_date, total_amount,quantity, customer_id, product_id

# customer_tbl
customer_id, name, email, address

# products_tbl
product_id, name, category, price
```

In [231…
```
order_id = []
order_date = []
total_amount = []
quantity = []
customer_id = []
product_id = []
product_name = []
category = []
price = []
customer_name = []
email = []
address = []
cust_id = []

for order in data:
        cust_id.append(order['customer']['customer_id'])
        customer_name.append(order['customer']['name'])
        email.append(order['customer']['email'])
        address.append(order['customer']['address'])
    for product in order['products']:
        order_id.append(order['order_id'])
        order_date.append(order['order_date'])
        customer_id.append(order['customer']['customer_id'])
        total_amount.append(order['total_amount'])
        quantity.append(product['quantity'])
        product_id.append(product['product_id'])
        product_name.append(product['name'])
        category.append(product['category'])
        price.append(product['price'])


df_orders_1 = pd.DataFrame(
    {'order_id' : order_id,
        'order_date' : order_date,
         'customer_id' : customer_id,
        'total_amount' : total_amount,
        'quantity' : quantity,
         'product_id' : product_id
    })

df_products = pd.DataFrame(
    {'product_id' : product_id,
        'product_name' : product_name,
```

```
        'category' : category,
        'price' : price
    })

df_customers = pd.DataFrame(
    {'cust_id' : cust_id,
        'customer_name' : customer_name,
        'email' : email,
        'address' : address
    })
```

Out[231…

	cust_id	customer_name	email	address
0	101	John Doe	johndoe@example.com	123 Main StSpringfield
1	102	Jane Smith	janesmith@example.com	456 Oak StSpringfield
2	103	Alice Johnson	alicejohnson@example.com	789 Birch StSpringfield
3	101	John Doe	johndoe@example.com	123 Main StSpringfield
4	104	Bob Brown	bobbrown@example.com	101 Maple StSpringfield
5	105	Sania	sania@example.com	102 Pine StSpringfield
6	106	Rohit	rohit@example.com	103 Cedar StSpringfield
7	107	Nina Patel	ninapatel@example.com	104 Birch StSpringfield
8	108	Tom Harris	tomharris@example.com	105 Oak StSpringfield
9	109	Eva Lee	evalee@example.com	106 Elm StSpringfield
10	101	John Doe	johndoe@example.com	123 Main StSpringfield
11	103	Alice Johnson	alicejohnson@example.com	789 Birch StSpringfield

In [224…

```
data
```

Out[224…


```
[{'order_id': 1,
  'order_date': '2024-01-10',
  'total_amount': 200.5,
  'customer': {'customer_id': 101,
   'name': 'John Doe',
   'email': 'johndoe@example.com',
   'address': '123 Main StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 2},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 1}]},
 {'order_id': 2,
  'order_date': '2024-01-12',
  'total_amount': 150.0,
  'customer': {'customer_id': 102,
   'name': 'Jane Smith',
   'email': 'janesmith@example.com',
   'address': '456 Oak StSpringfield'},
  'products': [{'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1}]},
 {'order_id': 3,
  'order_date': '2024-01-12',
  'total_amount': 120.0,
  'customer': {'customer_id': 103,
   'name': 'Alice Johnson',
   'email': 'alicejohnson@example.com',
   'address': '789 Birch StSpringfield'},
  'products': [{'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 2}]},
 {'order_id': 4,
  'order_date': '2024-01-13',
  'total_amount': 300.5,
  'customer': {'customer_id': 101,
   'name': 'John Doe',
   'email': 'johndoe@example.com',
   'address': '123 Main StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 3},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
```

```
   'price': 45.0,
   'quantity': 2}]},
 {'order_id': 5,
  'order_date': '2024-01-14',
  'total_amount': 180.0,
  'customer': {'customer_id': 104,
   'name': 'Bob Brown',
   'email': 'bobbrown@example.com',
   'address': '101 Maple StSpringfield'},
  'products': [{'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1},
   {'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 1}]},
 {'order_id': 6,
  'order_date': '2024-01-14',
  'total_amount': 250.0,
  'customer': {'customer_id': 105,
   'name': 'Sania',
   'email': 'sania@example.com',
   'address': '102 Pine StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 2},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 2},
   {'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 1}]},
 {'order_id': 7,
  'order_date': '2024-01-16',
  'total_amount': 180.0,
  'customer': {'customer_id': 106,
   'name': 'Rohit',
   'email': 'rohit@example.com',
   'address': '103 Cedar StSpringfield'},
  'products': [{'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 2},
   {'product_id': 'P05',
    'name': 'Desk Organizer',
    'category': 'Furniture',
```

```
   'price': 30.0,
   'quantity': 1}]},
 {'order_id': 8,
  'order_date': '2024-01-16',
  'total_amount': 230.0,
  'customer': {'customer_id': 107,
   'name': 'Nina Patel',
   'email': 'ninapatel@example.com',
   'address': '104 Birch StSpringfield'},
  'products': [{'product_id': 'P01',
    'name': 'Wireless Mouse',
    'category': 'Electronics',
    'price': 25.0,
    'quantity': 3},
   {'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1}]},
 {'order_id': 9,
  'order_date': '2024-01-17',
  'total_amount': 310.0,
  'customer': {'customer_id': 108,
   'name': 'Tom Harris',
   'email': 'tomharris@example.com',
   'address': '105 Oak StSpringfield'},
  'products': [{'product_id': 'P04',
    'name': 'Gaming Headset',
    'category': 'Electronics',
    'price': 60.0,
    'quantity': 2},
   {'product_id': 'P02',
    'name': 'Bluetooth Keyboard',
    'category': 'Electronics',
    'price': 45.0,
    'quantity': 2}]},
 {'order_id': 10,
  'order_date': '2024-01-18',
  'total_amount': 130.0,
  'customer': {'customer_id': 109,
   'name': 'Eva Lee',
   'email': 'evalee@example.com',
   'address': '106 Elm StSpringfield'},
  'products': [{'product_id': 'P05',
    'name': 'Desk Organizer',
    'category': 'Furniture',
    'price': 30.0,
    'quantity': 1},
   {'product_id': 'P03',
    'name': 'Laptop Stand',
    'category': 'Electronics',
    'price': 75.0,
    'quantity': 1}]},
 {'order_id': 11,
  'order_date': '2024-01-18',
  'total_amount': 250.0,
```

```
 'customer': {'customer_id': 101,
  'name': 'John Doe',
  'email': 'johndoe@example.com',
  'address': '123 Main StSpringfield'},
 'products': [{'product_id': 'P04',
   'name': 'Gaming Headset',
   'category': 'Electronics',
   'price': 60.0,
   'quantity': 2},
  {'product_id': 'P02',
   'name': 'Bluetooth Keyboard',
   'category': 'Electronics',
   'price': 45.0,
   'quantity': 2}]},
{'order_id': 12,
 'order_date': '2024-01-19',
 'total_amount': 180.0,
 'customer': {'customer_id': 103,
  'name': 'Alice Johnson',
  'email': 'alicejohnson@example.com',
  'address': '789 Birch StSpringfield'},
 'products': [{'product_id': 'P01',
   'name': 'Wireless Mouse',
   'category': 'Electronics',
   'price': 25.0,
   'quantity': 2},
  {'product_id': 'P05',
   'name': 'Desk Organizer',
   'category': 'Furniture',
   'price': 30.0,
   'quantity': 1}]}]
```

Handing de-duplication

In [236…

```
# drop duplicates based on latest information.

df_products.drop_duplicates(subset='product_id', keep='last', inplace=True)
```

In [237…

```
# drop duplicates based on latest information.

df_customers.drop_duplicates(subset='cust_id', keep='last', inplace= True)
```

In [238…

```
df_orders_1
```

Out[238...

	order_id	order_date	customer_id	total_amount	quantity	product_id
0	1	2024-01-10	101	200.5	2	P01
1	1	2024-01-10	101	200.5	1	P02
2	2	2024-01-12	102	150.0	1	P03
3	3	2024-01-12	103	120.0	2	P04
4	4	2024-01-13	101	300.5	3	P01
5	4	2024-01-13	101	300.5	2	P02
6	5	2024-01-14	104	180.0	1	P03
7	5	2024-01-14	104	180.0	1	P04
8	6	2024-01-14	105	250.0	2	P01
9	6	2024-01-14	105	250.0	2	P02
10	6	2024-01-14	105	250.0	1	P04
11	7	2024-01-16	106	180.0	2	P02
12	7	2024-01-16	106	180.0	1	P05
13	8	2024-01-16	107	230.0	3	P01
14	8	2024-01-16	107	230.0	1	P03
15	9	2024-01-17	108	310.0	2	P04
16	9	2024-01-17	108	310.0	2	P02
17	10	2024-01-18	109	130.0	1	P05
18	10	2024-01-18	109	130.0	1	P03
19	11	2024-01-18	101	250.0	2	P04
20	11	2024-01-18	101	250.0	2	P02
21	12	2024-01-19	103	180.0	2	P01
22	12	2024-01-19	103	180.0	1	P05